Politechnika Śląska
Wydział Matematyki Stosowanej
Kierunek Informatyka

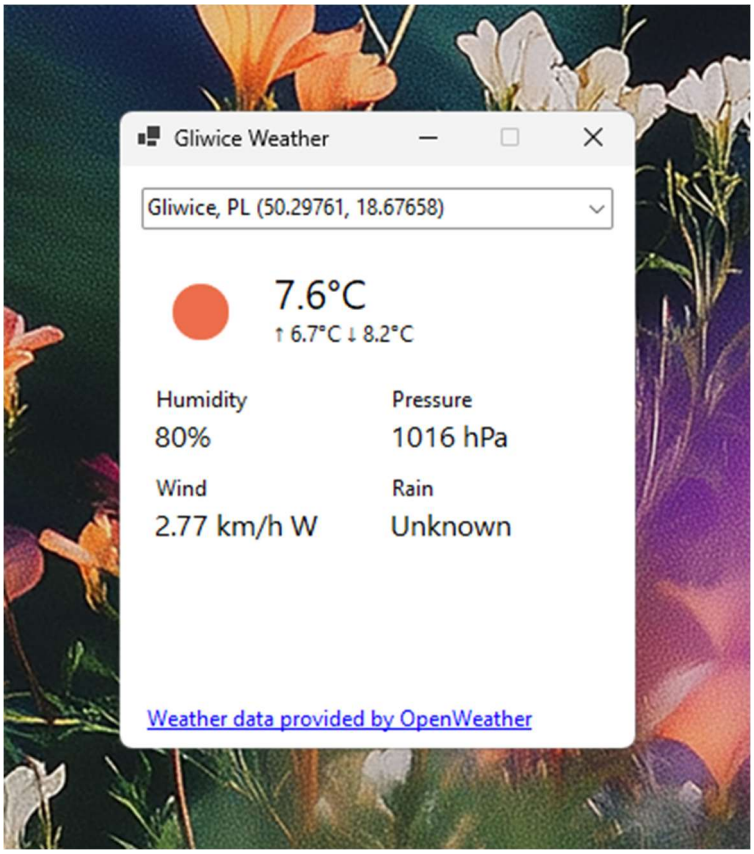
Gliwice, 24.03.2026

Programowanie obiektowe i graficzne

Miniprojekt 3

Aplikacja „Pogodynka”

Bartosz Bieniek gr. lab. 4



Zrzut ekranu 1 Główny ekran aplikacji.

Opis projektu.

Celem projektu było przygotowanie aplikacji desktopowej „Pogodynka” w technologii WinForms, pobierającej dane pogodowe z zewnętrznych API.

Wykorzystane technologie.

- Windows Forms
- C# 14
- .NET 10
- System.Net.Http
- System.Net.Json

Wykorzystane zewnętrzne API.

- OpenWeatherMap – Dane pogodowe dla wybranej lokalizacji.
- OpenDataSoft – Lista największych stolic europejskich oraz wyszukiwanie miast po nazwie.

Funkcjonalności.

- Użytkownik wyszukuje miasto po nazwie lub wybiera jedną z największych stolic europejskich z listy.
- Aplikacja wyświetla prognozę pogody dla wybranego miejsca, w tym temperaturę, wilgotność oraz ciśnienie atmosferyczne.
- Aplikacja wyświetla ikonę odpowiadającą obecnej prognozie.

Struktura projektu.

Application.Abstractions.Interfaces – Zbiór wspólnych interfejsów dla warstwy aplikacji.

- IRequest – Model żądania - polecenia lub zapytania (CQ).
- IRequestHandler – Reprezentuje obiekt obsługujący dane żądanie (wzorzec CQRS).

Application.Abstractions.Repositories – Interfejsy opisujące repozytoria.

- ICityRepository – Obsługuje wyszukiwanie miast.
- IForecastRepository – Wyszukiwanie prognozy pogody.

Application.Queries – Use-case’y obsługujące zapytania.

- GetCurrentWeather – Pobiera aktualną prognozę pogody dla wskazanej lokalizacji.
- GetLargestEuropeanCapitals – Pobiera listę największych stolic europejskich.
- GetCitiesByName – Wyszukuje miasta po nazwie (części nazwy).

Domain.Entities – Zawiera reprezentacje modeli domenowych.

- City – Miasto wraz z położeniem.
- Forecast – Prognoza pogody.
- Geolocation – Położenie fizyczne.

Infrastructure.OpenWeatherMap – Implementacja warstwy dostępu do danych dla API OpenWeatherMap (pobieranie danych pogodowych).

- Contracts.CurrentWeatherResponse – Model odpowiedzi z aktualną prognozą pogody zwracanej przez serwer.
- Mappers.CurrentWeatherResponseMapper – Mapuje odpowiedź serwera na domenowy model prognozy pogody.
- Mappers.ICurrentWeatherResponseMapper – Interfejs opisujący powyższy mapper, do potencjalnego wykorzystania podczas testowania.
- HttpOpenWeatherMapForecastRepository – Implementacja IForecastRepository, pobierająca dane pogodowe z internetowego API OpenWeatherMap.
- OpenWeatherMapOptions – Model opisujący sekcję ustawień w pliku appconfig.json dotyczących konfiguracji API OpenWeatherMap.

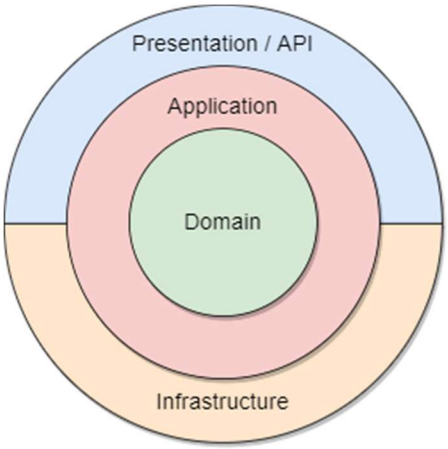
Infrastructure.OpenDataSoft – Analogiczna do powyższej implementacja warstwy dostępu do danych dla API OpenDataSoft (wyszukiwanie miast).

Presentation.Main – Implementacja warsty prezentacji dla głównego okna.

- Contracts.IMainPresenter – Interfejs opisujący prezenter głównego okna.
- Contracts.IMainView – Interfejs reprezentujący widok głównego okna.
- MainForm – Główne okno aplikacji.
- MainPresenter – Prezenter głównego okna.
- WeatherWidget – Pomocniczy komponent wyświetlający dane dotyczące jednego wskaźnika pogodowego.

Wzorce obiektowe.

Aplikacja została zaimplementowana w architekturze cebuli z wykorzystaniem wzorców MVP oraz CQRS.



Rysunek 1 Onion Architecture.
Źródło: https://github.com/NilavPatel/dotnet-onion-architecture

Projekt został podzielony na trzy główne warstwy.

- *Warstwa domenowa* reprezentuje modele oraz logikę biznesową aplikacji. Ze względu na prostotę projektu istnieją tylko te pierwsze, w których skład wchodzą definicje prognozy pogody, miasta oraz lokalizacji fizycznej.
- *Warstwa aplikacji* koordynuje przepływ danych w projekcie. Z założenia nie zna ona zasad biznesowych ani konkretnych implementacji warstwy danych, ale potrafi orkiestrować ich współdziałanie. Przykładowo, po otrzymaniu zapytania o listę najpopularniejszych stolic europejskich, odpytuje (przez interfejs) odpowiednie repozytorium, a następnie zwraca wyniki. Gdyby aplikacja wybierała jeden z kilku różnych serwisów pogodowych w zależności od ich obciążenia, to tutaj znajdowałyby się logika wyłaniająca i korzystająca z odpowiedniego z nich.
- *Warstwy prezentacji i infrastruktury* zawierają konkretne implementacje widoków i repozytoriów, zależne od wykorzystanych technologii. Odpowiadają one między innymi za interakcję z użytkownikiem, zapis danych oraz komunikację z zewnętrznymi API.

Wzorzec CQRS służy do rozdzielenia operacji modyfikacji danych od zapytań, co znacząco poprawia testowalność aplikacji oraz ułatwia jej późniejsze rozszerzanie. Polecenia i zapytania realizują tylko jedno zadanie – modyfikują lub odczytują dane, dzięki czemu nie powstają ogromne, trudne do mockowania serwisy. Gdyby do aplikacji miał zostać dodany nowy widok zawierający listę zapisanych miejscowości wraz z podsumowaniem pogody,

wystarczyłoby utworzyć odpowiednie zapytanie (Query), bez ingerencji w istniejące już serwisy.

Warstwa prezentacji odpowiedzialna jest za obsługę interfejsu graficznego aplikacji i została zaimplementowana z wykorzystaniem MVP. Wzorzec ten rozdziela aplikację na trzy części – modele, widoki oraz prezenter.

- Warstwa modeli odpowiada za logikę działania aplikacji, ale nie zawiera informacji o sposobie ich wyświetlania użytkownikowi. W projekcie tę rolę pełnią obiekty warstwy domenowej oraz aplikacji.
- Widoki zajmują się prezentacją oraz pobieraniem danych od użytkownika, stąd ich implementacja zależy w dużej mierze od wykorzystanych bibliotek, na przykład Windows Forms.
- Rolę pośrednika między wyżej wspomnianymi warstwami pełnią prezentery. Odbierają one dane z interfejsu użytkownika, wywołują odpowiednie polecenia i zapytania na warstwie modeli, po czym przekazują wyniki z powrotem do widoku. Warto zaznaczyć, że nie powinny one być jakkolwiek zależne od wybranej technologii wykorzystywanej w widoku – dane przesyłane przez zdarzenia muszą być więc odpowiednio wcześniej w widoku przygotowane.

Widoki nie powinny oczywiście zawierać mechanizmów biznesowych, ale istnieją dwa sposoby ich implementacji we wzorcu MVP. Pierwszym z nich jest „*passive view*”, w którym to widoki praktycznie nie zawierają „*code behind*” i wszystko przekazują do prezenterów (rzecz jasna w formacie pozbawionym zależności od bibliotek interfejsu użytkownika). Wariant „*supervising controller*”, z którego skorzystałem w projekcie, zezwala natomiast na zawarcie pewnej części logiki wewnątrz widoku, na przykład *bindingów*.

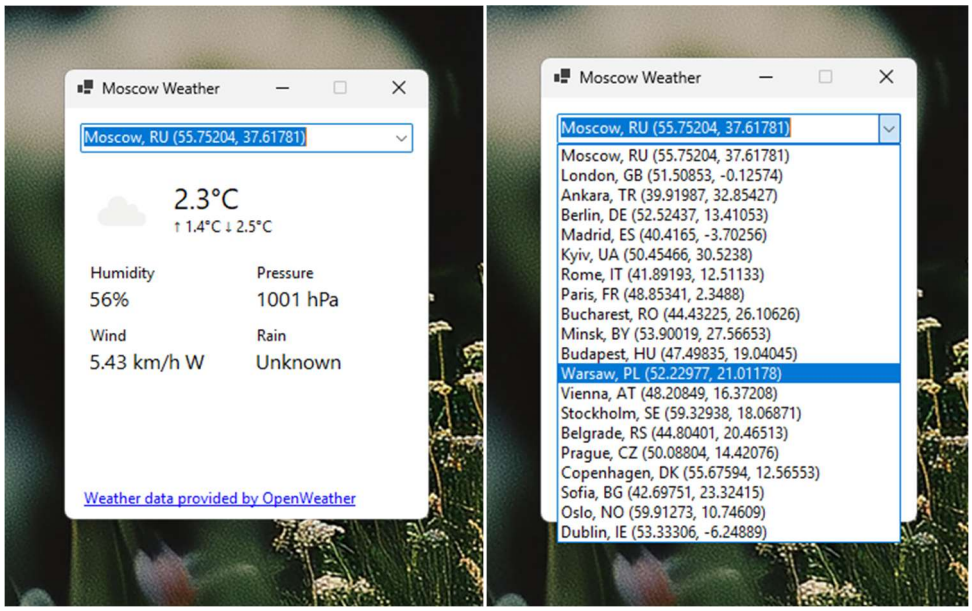
Przykładowy przepływ danych w mojej aplikacji wygląda następująco.

1. Użytkownik wybiera z listy miasto, dla którego chce sprawdzić prognozę pogody.
2. Widok odbiera zdarzenie o wyborze pozycji z listy, sprawdza zaznaczoną opcję, a następnie generuje nowe zdarzenie wyboru miasta.
3. Prezenter odbiera zdarzenie oraz tymczasowo czyści aktualną prognozę pogody w widoku. Na podstawie otrzymanych danych tworzy obiekt zapytania, który wysyła do odpowiedniego use-case w warstwie aplikacji.
4. Use-case odbiera dane, po czym (przez interfejs) pobiera dane pogodowe z odpowiedniego repozytorium
5. Implementacja repozytorium w warstwie infrastruktury wykonuje zapytanie do API z danymi pogodowymi, mapuje otrzymaną odpowiedź do właściwego formatu, a następnie zwraca dane.
6. Use-case odbiera dane i przekazuje je do prezentera.

7. Prezenter odbiera dane i przekazuje je do widoku.
8. Widok na podstawie otrzymanych danych pogodowych aktualizuje zawartość odpowiednich pól tekstowych, wybiera odpowiedni obrazek oraz zmienia tytuł okna.
9. Użytkownikowi wyświetlona zostaje prognoza pogody, dla wybranego miejsca.

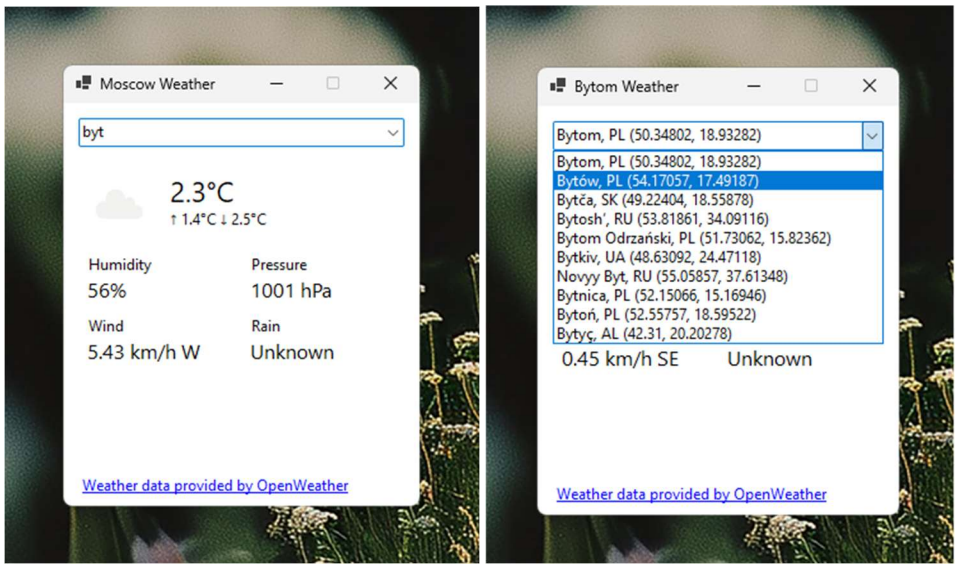
Instrukcja użytkownika.

Po otwarciu aplikacji automatycznie zostanie pobrana prognoza pogody dla jednej z europejskich stolic.



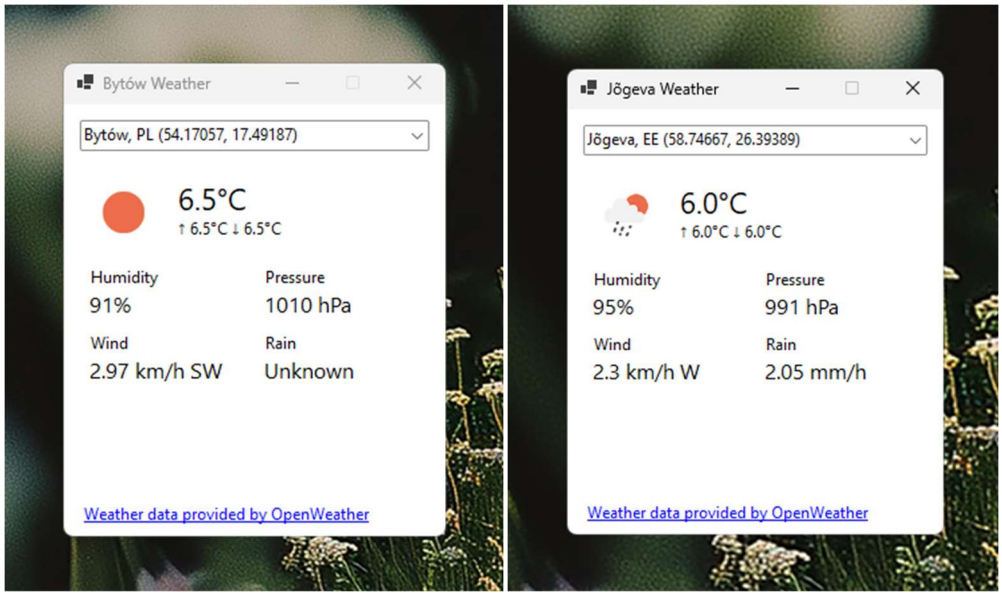
Zrzut ekranu 2 (Po lewej) Automatycznie załadowana prognoza pogody.
(Po prawej) Wybór jednej z największych stolic europejskich.

Rozwijając listę można dokonać wyboru jednej z największych stolic europejskich, dla której zostanie załadowana prognoza pogody. W polu tekstowym można także wpisać nazwę (część nazwy) dowolnego innego miasta, a następnie wybrać je z listy wyników.



Zrzut ekranu 3 Wyszukiwanie miasta po nazwie.

Dla zaznaczonej lokalizacji zostanie oczywiście wyświetlona aktualna prognoza pogody.



Zrzut ekranu 4 Prognoza pogody dla wybranego miasta.

Usuwając cały tekst z pola wyszukiwania, można ponownie zobaczyć listę największych stolic europejskich.